WERGONIC · ENGINEERING

Technical Documentation

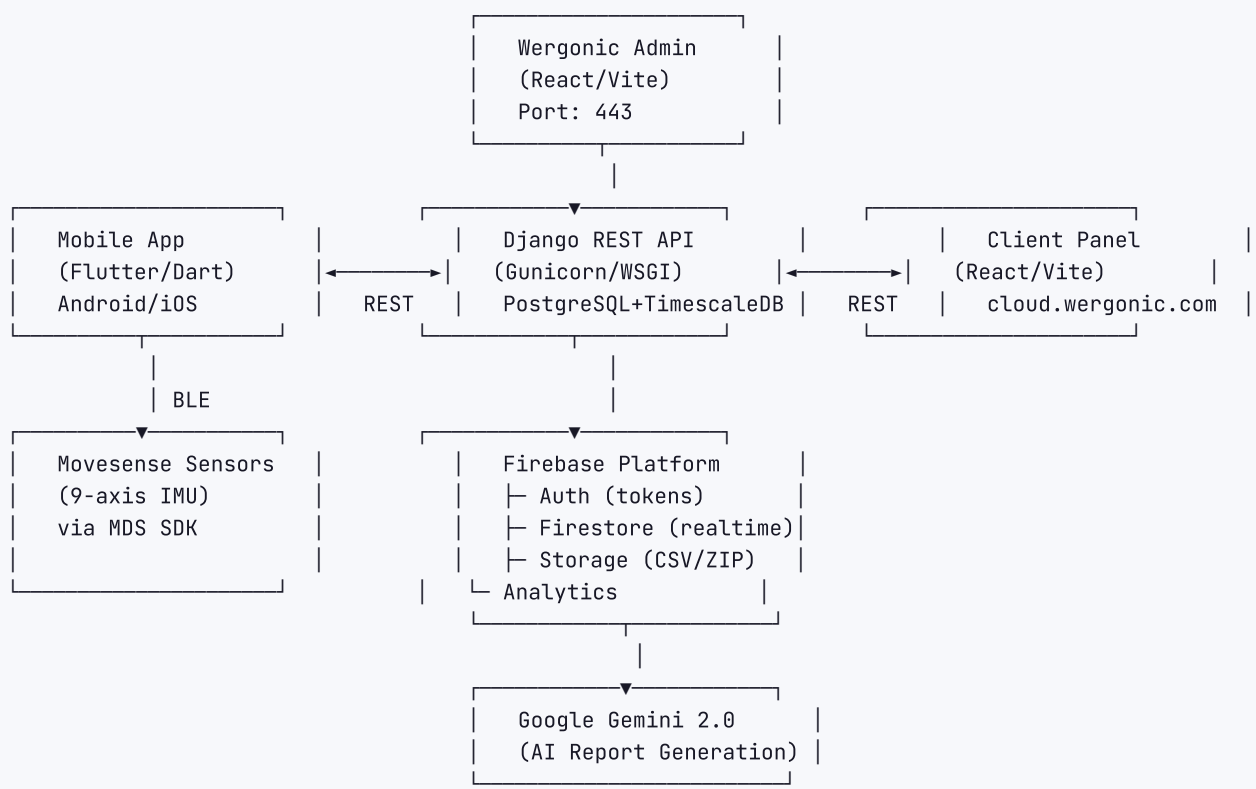
System architecture, data models, API surface, mobile and web internals, the end-to-end data pipeline, and the technical debt register — reconstructed from the codebase.

DOCUMENT	GENERATED	SOURCE
Engineering reference	17 March 2026	Branch `dev` — all 3 repositories

Contents

Architecture Overview

High-Level System Diagram



Data Flow:

1. Movesense sensors → BLE → Mobile App (Flutter)
2. Mobile App → processes IMU data → writes CSV files locally
3. Mobile App → uploads ZIP (CSV files) → Firebase Storage
4. Mobile App → creates/updates session → Django REST API
5. Django Backend → fetches CSV from Firebase Storage → parses → stores in TimescaleDB
6. Django Backend → computes stats, assessments, TNO analysis
7. Django Backend → sends session data to Gemini → receives AI analysis
8. Web Client Panel → fetches processed data → displays charts/tables/reports
9. Firestore used for real-time session status + push notifications

Tech Stack

COMPONENT	TECHNOLOGY	VERSION
Backend	Django + DRF	5.0.9 / 3.15.2
Database	PostgreSQL + TimescaleDB	—
Frontend	React + TypeScript + Vite	18.2.0 / 4.9.3 / 4.1.1
Mobile	Flutter/Dart	SDK ≥3.3.1
State (Web)	Jotai + TanStack React Query	2.0.2 / 4.24.10
State (Mobile)	BLoC/Cubit (flutter_bloc)	8.1.2
UI (Web)	Material-UI 5 + Emotion	5.11.9
Charts (Web)	Nivo	0.83.0
BLE	flutter_blue_plus + MDS SDK	1.35.3 / 2.3.1
Local DB (Mobile)	Drift (SQLite)	2.28.2
Auth	Firebase Auth	9.17.1 (web) / latest (mobile)
Storage	Firebase Storage	—
Realtime	Cloud Firestore	—
AI	Google Generative AI (Gemini)	0.7.2
Error Tracking	Sentry	7.40.0 (web) / 8.14.1 (mobile)
Monorepo (Web)	Turborepo + Yarn	1.8.1 / 1.22.19
WSGI Server	Gunicorn	22.0.0

Deployment Architecture

- **Backend:** Gunicorn WSGI server, PostgreSQL with TimescaleDB extension
- **Web Apps:** Vite-built static sites, Docker containerized (ports 443, 8443)
- **Mobile:** Flutter app, built via Codemagic CI/CD
- **Firebase:** Auth, Firestore, Storage, Analytics, Crashlytics — Google Cloud hosted
- **Error Tracking:** Sentry (separate DSNs for web and mobile)
- **Environment tiers:** DEV, STAGING, PROD — each with own database and Firebase project

Environment Configuration

Backend uses `python-decouple` with environment-prefixed variables:

- `{ENVIRONMENT}_DATABASE_NAME/USER/PASSWORD/HOST/PORT` — per-tier DB config
- `{ENVIRONMENT}_project_id`, `private_key`, etc. — per-tier Firebase credentials
- `{ENVIRONMENT}_FRONTEND_BASE_URL` — per-tier frontend URL for magic links

- `GEMINI_API_KEY` — Google Generative AI key
- `sentry_dsn` — Error tracking
- `slack_token` — Slack notifications
- `EMAIL_HOST/USER/PASSWORD` — SMTP (port 465, SSL)

Web apps use Vite env vars:

- `VITE_API_BASE_URl` + `VITE_API_VERSION` — Backend API URL
- `VITE_API_KEY`, `VITE_AUTH_DOMAIN`, `VITE_PROJECT_ID`, `VITE_STORAGE_BUCKET` — Firebase config
- `VITE_RECAPTCHA_SITE_KEY` — reCAPTCHA v3 for AppCheck
- `MODE` — development/production

Backend (Django)

2.1 Project Structure

```
wergonic-django-backend/
├── core/                      # Main Django project
│   ├── settings.py            # Configuration (env-aware)
│   ├── urls.py                # Root URL routing
│   ├── cdn/conf.py            # Firebase Storage/CDN config
│   ├── pagination.py          # DRF pagination (PAGE_SIZE=20)
│   └── test_setup.py          # Test fixtures
├── users/                     # User management app
├── firebase_auth/             # Firebase authentication backend
├── user_sessions/             # Session recording & calculations
├── organizations/             # Organization management
├── ramp/                      # RAMP/RULA/MEC/REBA assessments
├── devices/                   # Device & sensor management
├── tickets/                   # Support tickets
├── work_tags/                 # Work task/label taxonomy (MPTT)
├── tno/                       # Task/movement analysis (TNO)
├── notifications/             # User notifications
├── workcycles/                # Factory hierarchy
├── utils/                     # Shared utilities
└── requirements.txt           # 166 dependencies
```

App Responsibilities

APP	PURPOSE
`users`	Custom User model (AbstractBaseUser), Profile, UserOrganization, Invitation, registration, password management
`firebase_auth`	Firebase ID token verification DRF authentication backend
`user_sessions`	Session model, Measurement/MeasurementData (TimescaleDB), HRData, SessionStats, Log, AIRequestLog, session lifecycle, CSV processing, stats calculation
`organizations`	Organization model, org CRUD, user-org management, stats, invitations
`ramp`	RAMPAssessment model (covers RAMP, RULA, REBA, MEC), assessment CRUD
`devices`	Device, Sensor, Firmware models and management
`tickets`	Ticket model for support requests
`work_tags`	Category (MPTT tree), Label, Event models — hierarchical tagging system
`tno`	TNO calculation engine — joint angle analysis, peak classification
`notifications`	Notification model + Firestore push on creation
`workcycles`	Factory → Line → Workstation → TaskModel → Task hierarchy
`utils`	Shared utilities, search filters

2.2 Data Models (Complete Schema)

Users Domain

USER (CUSTOM ABSTRACTBASEUSER)

Field	Type	Constraints
email	EmailField	unique
user_name	CharField(150)	unique (Firebase UID)
first_name	CharField(50)	
last_name	CharField(50)	
unit	CharField	nullable
image	URLField	nullable
start_date	DateTimeField	default=now
role	CharField	choices=[WERGONIC_ADMIN, ORG_ADMIN, ERGONOMIST, WORKER]
personal_number	CharField	nullable
subject_ID	PositiveIntegerField	nullable
is_active	BooleanField	default=True
is_verified	BooleanField	default=False
is_staff	BooleanField	default=False
user_locale	CharField	default='en'
organizations	ManyToMany(Organization)	via UserOrganization
organization	ForeignKey(Organization)	nullable (current/primary)
last_seen	DateTimeField	nullable
ai_consent	BooleanField	default=False
ramp_limit_display	CharField	choices=[TIME, PERCENTAGE]

Custom Manager: `UserManager` — `create_user()`, `create_superuser()` **Signals:** `pre_save` creates Firebase user + sets custom claims; `pre_delete` deletes Firebase user

PROFILE

Field	Type	Constraints
user	ForeignKey(User)	
consultant	ForeignKey(User)	nullable (assigned ergonomist)
weight	CharField	
resting_heart_rate	CharField	
gender	CharField	choices=[MALE, FEMALE, OTHER]
dominant_arm	CharField	choices=[RIGHT, LEFT]
date_of_birth	DateField	nullable
age	CharField	
comment	CharField	
first_name	CharField	
last_name	CharField	
unit	CharField	
image	URLField	nullable
personal_number	CharField	nullable
is_main	BooleanField	default=False

USERORGANIZATION

Field	Type	Constraints
user	ForeignKey(User)	
organization	ForeignKey(Organization)	
role	CharField	choices from User.Role
is_active	BooleanField	
is_deactivated	BooleanField	

Constraint: `unique_together = (user, organization)`

INVITATION

Field	Type	Constraints
sender	ForeignKey(User)	
recipient	ForeignKey(User)	
organization	ForeignKey(Organization)	
role	CharField	
is_accepted	BooleanField	default=False
expiration_date	DateTimeField	default=now + 7 days

Organizations Domain

ORGANIZATION

Field	Type	Constraints
name	CharField	unique
country	CharField	nullable
address	CharField	nullable
type	CharField	choices=[Company, Private]
industry	CharField	choices=[A-U industry codes]
cnr	CharField	nullable
contact_phone	CharField	nullable
max_number_workers	PositiveIntegerField	default=500
max_number_admins	PositiveIntegerField	default=5
max_number_sessions_month	PositiveIntegerField	default=3000
max_number_assessments_month	PositiveIntegerField	default=3000
logo	URLField	nullable
current_subscription	CharField	
referral_source	CharField	
license_expiration	DateField	nullable
is_archived	BooleanField	
is_active	BooleanField	
ai_engine	CharField	choices=[Gemini, Chat-button-001], default=Gemini
hw_kit_access	CharField	choices=[OWN, BORROWED, NO_ACCESS]

Computed Properties: `employees`, `number_of_employees`, `number_of_admins`, `number_of_sessions_per_month`, `number_of_assessments_per_month`

Sessions Domain

SESSION

Field	Type	Constraints
started_at	DateTimeField	
ended_at	DateTimeField	
updated_at	DateTimeField	auto_now
status	CharField	choices=[ONGOING, PAUSED, FAILED, FINISHED, INTERRUP
worker	ForeignKey(Profile)	
recorded_by	ForeignKey(Profile)	(the ergonomist/admin)
organization	ForeignKey(Organization)	
tag	CharField	
assessment_type	CharField	
workplace	CharField	
mobile_app_version	CharField	
measurments_csv	URLField	(Firebase Storage URL)
duration	PositiveBigIntegerField	(seconds)
max_duration_minutes	IntegerField	default=480
devices_json	JSONField	
calculations	JSONField	(posture/movement stats)
tno_calculations	JSONField	(TNO analysis results)
temperature_calculations	JSONField	
heart_rate_calculations	JSONField	(HRV analysis)
tasks_percentiles_calculations	JSONField	
tasks	JSONField	(event list)
calculation_details	JSONField	
ai_report	JSONField	(Gemini analysis)
calculations_completed	BooleanField	
processing	BooleanField	
is_new_calculations	BooleanField	
has_all_limbs	BooleanField	
workcycle_task	ForeignKey(Task)	nullable

Signals: `post_save` triggers async calculation pipeline when FINISHED; `pre_save` updates duration, sets status, deletes Firestore doc

MEASUREMENT

Field	Type	Constraints
session	ForeignKey(Session)	
name	CharField	(limb name)
environment	CharField	
version	CharField	

MEASUREMENTDATA (TIMESCALEDB HYPERTABLE)

Field	Type	Constraints
measurement	ForeignKey(Measurement)	
event	ForeignKey(Event)	nullable
time	TimescaleDateTimeField	interval="8 hours"
timestamp_node	IntegerField	
timestamp_android	IntegerField	
time_delta	FloatField	
acc_x	FloatField	accelerometer X
acc_y	FloatField	accelerometer Y
acc_z	FloatField	accelerometer Z
gyro_x	FloatField	gyroscope X
gyro_y	FloatField	gyroscope Y
gyro_z	FloatField	gyroscope Z
elevation_angle	FloatField	
angular_velocity	FloatField	
gyro_angular_velocity	FloatField	
side_bending	FloatField	
forward_bending	FloatField	
flexion	FloatField	
abduction	FloatField	
pitch	FloatField	
temperature	FloatField	

Managers: `objects` (default), `timescale` (TimescaleManager)

HRDATA (TIMESCALEDB HYPERTABLE)

Field	Type	Constraints
measurement	ForeignKey(Measurement)	
event	ForeignKey(Event)	nullable
time	TimescaleDateTimeField	
time_delta	PositiveIntegerField	
heart_rate	FloatField	
rr	FloatField	(R-R interval)
heart_rate_reserve	FloatField	

SESSIONSTATS

Field	Type	Constraints
session	ForeignKey(Session)	
posture	ManyToMany(PostureData)	
speed_of_movement	ManyToMany(SpeedData)	
wrist_distance	ManyToMany(WristData)	

Method: `get_zone_counts()` — aggregate wrist distance zones

LOG

Field	Type	Constraints
created_at	DateTimeField	default=now
session	ForeignKey(Session)	
description	CharField	
media	CharField	(Firebase Storage path)

AIREQUESTLOG

Field	Type	Constraints
organization	ForeignKey(Organization)	
timestamp	DateTimeField	default=now

Assessments Domain

RAMPASSESSMENT

Field	Type	Constraints
assessment_name	CharField	
worker	CharField	
work_task	CharField	
date	DateField	
assessment_type	CharField	
site	CharField	nullable
country	CharField	nullable
unit	CharField	nullable
company_representative	CharField	nullable
assessment_completed_by	CharField	
comment	TextField	
assessment	JSONField	(actual RAMP/RULA/MEC/REBA data)
status	CharField	choices=[COMPLETED, IN_PROGRESS, PROCESSING]
assesment_category	CharField	choices=[RAMP, RULA, MEC, REBA]
results_comment	CharField	
source	CharField	choices=[generated, manual]
created_by	ForeignKey(User)	
organization	ForeignKey(Organization)	
session	ForeignKey(Session)	nullable
start_time	TimeField	nullable
end_time	TimeField	nullable

Devices Domain

DEVICE

```
Field             Type                    Constraints
──────────────────────────────────────────────────────
created_at        DateTimeField
serial            CharField               unique
firmware_version  CharField
organization      ForeignKey(Organization) nullable
worker            ForeignKey(User)        nullable
locale            CharField
device_model      CharField
sensors           ManyToMany(Sensor)
```

SENSOR

```
Field       Type        Constraints
──────────────────────────────────
sensor_id   CharField   unique
limb        CharField   nullable
```

FIRMWARE

```
Field             Type          Constraints
──────────────────────────────────────────
description       TextField
mobile_version    CharField
release_date      DateTimeField
file_size         CharField
firmware_version  CharField
firmware_url      URLField
is_current        BooleanField  default=False
```

Work Tags Domain (MPTT)

CATEGORY (MPTTMODEL)

```
Field          Type                      Constraints
──────────────────────────────────────────────────
parent         TreeForeignKey(self)       nullable
organization   ForeignKey(Organization)
category_name  CharField
category_type  CharField                 choices=[WORK_TASK, WORK_LABEL]
created_at     DateTimeField
```

Meta: `order_insertion_by = ["-created_at"]`

LABEL

Field	Type	Constraints
label_name	CharField	
label_category	ForeignKey(Category)	
is_private	BooleanField	
owner	ForeignKey(User)	
favorited_by	ManyToMany(User)	
repeat_interval	CharField	nullable
created_at	DateTimeField	

EVENT

Field	Type	Constraints
session	ForeignKey(Session)	
start_time	TimeField	nullable
end_time	TimeField	nullable
labels	ManyToMany(Label)	
created_by	ForeignKey(User)	

Workcycles Domain

FACTORY

Field	Type	Constraints
name	CharField	
organization	ForeignKey(Organization)	

Constraint: Unique name per organization

LINE

Field	Type	Constraints
name	CharField	
factory	ForeignKey(Factory)	

Constraint: Unique name per factory

WORKSTATION

Field	Type	Constraints
name	CharField	
line	ForeignKey(Line)	

Constraint: Unique name per line

TASKMODEL

Field	Type	Constraints
name	CharField	
workstation	ForeignKey(Workstation)	

Constraint: Unique name per workstation

TASK

Field	Type	Constraints
name	CharField	
task_model	ForeignKey(TaskModel)	
description	TextField	nullable
duration	FloatField	nullable
created_at	DateTimeField	

Constraint: Unique name per task_model **Meta:** `ordering = ["-created_at"]`

Notifications Domain

NOTIFICATION

Field	Type	Constraints
recipient	ForeignKey(User)	
title	CharField	
message	TextField	
created_at	DateTimeField	auto_now_add
read_at	DateTimeField	nullable
notification_type	CharField	choices=[MEC_ASSESSMENT, SESSION, RAMP_ASSESSMENT]
organization	ForeignKey(Organization)	nullable
foreign_key	PositiveIntegerField	nullable (ID of related object)

Signal: `post_save` pushes to Firestore `organizations/{orgId}/users/{userId}/notifications/`

Tickets Domain

TICKET

Field	Type	Constraints
created_at	DateTimeField	
sender	ForeignKey(User)	
subject	CharField	
type	CharField	choices=[bug, feature, suggestion, help]
help_type	CharField	choices=[account, billing, other], nullable
message	TextField	
attachments	CharField	

2.3 API Endpoints (Complete Map)

Base URL: `/api/v1/`

Users (`/users/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/users/`	Auth	List all users / Create user
GET	`/users/admins/`	Auth	List org admins
GET, PUT, DELETE	`/users/<id>/`	Auth	CRUD single user
GET	`/users/current/`	Auth	Get authenticated user with full display data
POST	`/users/password/reset/`	Auth	Reset password (updates Firebase + Django)
POST	`/users/auth/passwordless-signin/`	Public	Send email magic link (web)
POST	`/users/auth/mobile-passwordless-signin/`	Public	Send email magic link (mobile)
POST	`/users/superuser/`	Auth	Create admin user
POST	`/users/email/check/`	Public	Check if email exists
POST	`/users/personal_number/check/`	Public	Check if personal number exists
GET	`/users/organizations/`	Auth	List user's organizations
POST	`/users/organizations/<orgId>/change-to-active/`	Auth	Switch active organization
POST	`/users/create-worker-account/`	Public	Register worker account

Organizations (`/organizations/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/organizations/`	LevelZero	List/Create organization
GET	`/organizations/names`	Auth	Get all org names
GET, PUT, DELETE	`/organizations/<id>/`	Auth	CRUD organization
GET	`/organizations/stats/`	LevelOne	Organization statistics (date-filtered)
GET, POST	`/organizations/users/`	Auth	List/create org users
GET, PUT, DELETE	`/organizations/users/<userId>/`	Auth	CRUD org user
POST	`/organizations/users/<userId>/deactivate/`	Auth	Deactivate user from org
GET, POST	`/organizations/users/<userId>/profiles/`	Auth	Manage user profiles
GET	`/organizations/profiles/`	Auth	List all profiles in org
POST	`/organizations/invitation/<invitationId>/`	Auth	Accept invitation
POST	`/organizations/change-to-archive/`	Auth	Archive/unarchive orgs
GET	`/organizations/<orgId>/ai-requests-per-month/`	Auth	AI usage stats
POST	`/organizations/register/`	Public	Register new org + admin

Sessions (`/sessions/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/sessions/organizations/<orgId>/`	Auth	Create/list sessions
GET, PUT, DELETE	`/sessions/<id>/`	SessionCrud	Update/view/delete session
GET	`/sessions/<id>/stats/`	SessionCrud	Session statistics
POST	`/sessions/stats/compare/`	Auth	Compare multiple sessions
POST	`/sessions/stats/compare/excel/`	Auth	Export comparison Excel
GET	`/sessions/<id>/stats/tno/`	SessionCrud	TNO analysis
GET	`/sessions/<id>/stats/temperature/`	SessionCrud	Temperature data
GET	`/sessions/<id>/stats/hr/`	SessionCrud	HR/HRV data
GET	`/sessions/<id>/stats/tasks/`	SessionCrud	Task percentiles
GET	`/sessions/<id>/stats/details/`	SessionCrud	Detailed stats
GET, POST	`/sessions/<id>/logs/`	SessionCrud	Upload/list logs
GET, PUT, DELETE	`/sessions/<sessionId>/logs/<id>/`	SessionCrud	CRUD log entry
POST	`/sessions/<id>/stats/export/`	SessionCrud	Export raw CSV
POST	`/sessions/<id>/stats/export/excel/`	SessionCrud	Export Excel
POST	`/sessions/<id>/stats/recalculate/`	SessionCrud	Recalculate stats
POST	`/sessions/<sessionId>/assesments/mappings/`	Auth	Generate MEC assessment
POST	`/sessions/merge/`	Auth	Merge sessions
DELETE	`/sessions/delete/<sessionId>/`	SessionCrud	Delete session
POST	`/sessions/<id>/generate-gemini-analysis/`	Auth	Generate AI report

Assessments (`/assesments/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/assesments/`	Auth	List/create assessment
GET, PUT, DELETE	`/assesments/<id>/`	Auth	CRUD assessment
POST	`/assesments/multi/`	Auth	Fetch multiple by IDs

Devices (`/devices/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/devices/`	Auth	List/create device
GET, PUT, DELETE	`/devices/<id>/`	AdminOrReadOnly	CRUD device
GET, POST	`/devices/firmwares/`	AdminOrReadOnly	List/create firmware
PUT, PATCH	`/devices/firmwares/<id>/`	AdminOrReadOnly	Update firmware
GET, POST	`/devices/sensors/`	Auth	List/create sensor
GET, PUT, DELETE	`/devices/sensors/<id>/`	AdminOrReadOnly	CRUD sensor

Work Tags (`/categories/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/categories/organizations/<orgId>/`	Auth	List/create categories
GET	`/categories/organizations/<orgId>/labels/`	Auth	List org labels
GET, POST	`/categories/organizations/<orgId>/category-with-labels/`	Auth	Nested category+labels
GET, PATCH	`/categories/organizations/<orgId>/category-with-labels/<id>/`	Auth	Update nested
GET, DELETE	`/categories/<id>/organizations/<orgId>/`	Auth	Category detail
GET, POST	`/categories/<categoryId>/labels/`	Auth	List/create labels
GET, POST	`/categories/labels/events/sessions/<sessionId>/`	Auth	List/create events
GET, PUT, DELETE	`/categories/labels/<id>/`	Auth	CRUD label
GET, PUT, DELETE	`/categories/labels/events/<id>/`	Auth	CRUD event

Work Cycles (`/workcycles/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/workcycles/factories/`	Auth	List/create factories
GET, PUT, PATCH, DELETE	`/workcycles/factories/<id>/`	Auth	CRUD factory
GET	`/workcycles/factories/full-hierarchy/`	Auth	Full tree
GET, POST	`/workcycles/lines/`	Auth	List/create lines
GET, PUT, PATCH, DELETE	`/workcycles/lines/<id>/`	Auth	CRUD line
GET, POST	`/workcycles/workstations/`	Auth	List/create workstations
GET, PUT, PATCH, DELETE	`/workcycles/workstations/<id>/`	Auth	CRUD workstation
GET, POST	`/workcycles/models/`	Auth	List/create task models
GET, PUT, PATCH, DELETE	`/workcycles/models/<id>/`	Auth	CRUD task model
GET, POST	`/workcycles/tasks/`	Auth	List/create tasks
GET, PUT, PATCH, DELETE	`/workcycles/tasks/<id>/`	Auth	CRUD task
POST	`/workcycles/import/`	Auth	Import work cycles (AVIX)

Notifications (`/notifications/`)

METHOD	PATH	AUTH	PURPOSE
GET	`/notifications/`	Auth	List notifications (paginated)
PUT, DELETE	`/notifications/<id>/`	Auth	Update/delete notification
POST	`/notifications/mark-as-read/`	Auth	Mark all as read

TNO (`/tno/`)

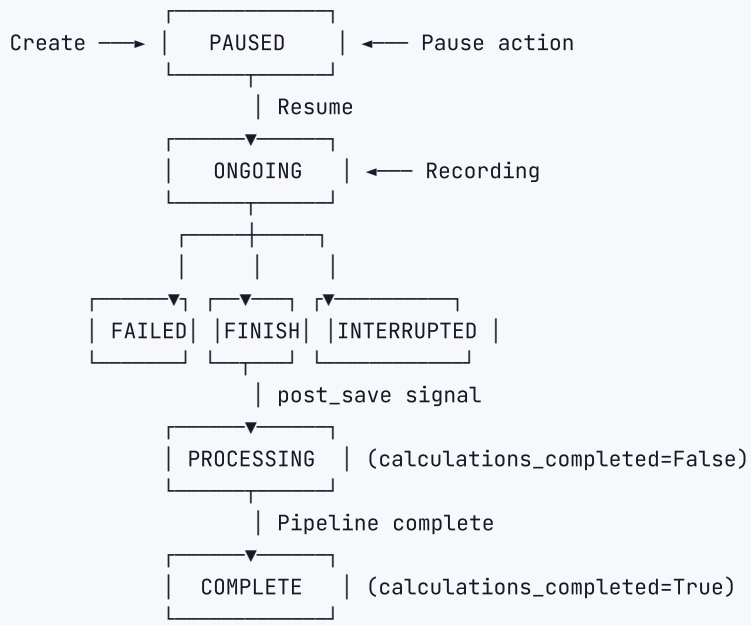
METHOD	PATH	AUTH	PURPOSE
POST	`/tno/`	Auth	Calculate TNO (accepts ZIP with CSV)

Tickets (`/tickets/`)

METHOD	PATH	AUTH	PURPOSE
GET, POST	`/tickets/`	Auth	List/create tickets
GET, DELETE	`/tickets/<id>/`	Auth	View/delete ticket

2.4 Business Logic & Services

Session Lifecycle (State Machine)



> `flag: Session status transitions are not enforced in code. Any status can be set to any other status via API.`

Data Processing Pipeline (Async)

Triggered by `Session_post_save` signal when `status=FINISHED` and `calculations_completed=False`:

Implementation: Spawns a Python `Thread` (not Celery/task queue).

Steps (sequential):

1. **CSV Processing** (`process_csv_and_create_measurement`):
   - Fetch ZIP from Firebase Storage at `Session.measurments_csv`
   - Extract CSV files (one per limb)
   - Parse columns: time, acc_x/y/z, gyro_x/y/z, angles, temperature
   - Create Measurement record per limb
   - Bulk create MeasurementData records in TimescaleDB (interval="8 hours")
   - Link to events if `session.tasks` provided
2. **Posture/Pie Data** (`Calculate_pie_data`):
   - Compute posture percentages per limb
   - Movement range distribution
   - Stored in `session.calculations` JSONField
3. **Temperature Data** (`Calculate_temperature_data`):
   - Mean, max, min, median temperature per limb
   - Global + per-event aggregation
   - Stored in `session.temperature_calculations`
4. **HR/HRV Analysis** (`Calculate_hr_data`):

- Uses `hrvanalysis` library
- Time domain: SDNN, RMSSD
- Frequency domain: LF/HF ratio
- Geometrical features
- CSI/CVI features (Cardiac Sympathetic/Vagal Index)
- Stored in `session.heart_rate_calculations`

5. **Chart Data** (`Calculate_charts_data`):
   - Posture line data (elevation angle, bending over time)
   - Speed of movement (angular velocity distributions)
   - Wrist distance zones (<30cm, 30-50cm, >50cm)
   - Creates SessionStats record with M2M relationships
6. **TNO Calculation** (`Calculate_tno_data`):
   - Joint angle analysis per limb
   - Peak classification (find movement peaks)
   - Static vs dynamic movement characterization
   - Time in angle ranges: >60°, 20-60°, -10-20°, <-10°
   - Static duration threshold: 6 seconds
   - Angle threshold: 5 degrees
   - Stored in `session.tno_calculations`
7. **Email Notification** (`stats_end_notification`):
   - Sends completion email to session recorder
   - Includes elapsed processing time

> `flag: Uses Thread-based async without retry, dead-letter, or monitoring. If processing fails, session stays unprocessed with no recovery mechanism.`

MEC Assessment Calculation

CreateAssessmentView generates RAMPAssessment from session sensor data:

1. Fetch MeasurementData for the specified time range (start_time → end_time)
2. **Arm Posture Analysis** (`analyze_arm_postures`):
   - Frequency analysis of dominant arm + forearm movements
   - Classify into elevation ranges
3. **Wrist Position Calculation** (`calculate_wrist_position`):
   - 2D geometric calculation from arm and forearm angles
   - Uses arm length assumptions
4. **Work Posture Models** (6 models):
   - `calculate_postures_1` through `calculate_postures_6`
   - Each handles different arm elevation ranges
   - Computes time-weighted risk scores
5. Generate assessment JSON with risk scores per posture
6. Create RAMPAssessment with `status=COMPLETED` , `assesment_category=MEC`

TNO Calculation Engine

`tno_calculate()` in `tno/` app:

1. **Peak Classification** — Find movement peaks in angle time series
2. **Movement Characteristics:**
   - Static duration (time held at one angle)
   - Upward/downward movement tracking
3. **Task Characteristics per angle range:**
   - static_>60°, static_20-60°, static_-10-20°, static_<-10°
   - dynamic_>60°, dynamic_20-60°, dynamic_-10-20°, dynamic_<-10°
4. **Constants:**
   - staticDurThreshold = 6 seconds
   - angleThreshold = 5 degrees
   - Angle boundaries: [60, 20, 10] degrees

AI Report Generation (Gemini)

Pipeline:

1. `GenerateGeminiAnalysisView` receives POST with language + optional tags
2. Calls `get_gemini_analysis()` with session data
3. Constructs prompt from `prompts.py`:

Prompt Structure:

- Role: "You are an ergonomist and H&S expert"
- Worker context: age, weight, dominant arm
- Duration context with extrapolation caveat
- Full session measurements JSON
- Risk zone thresholds:
  - ARM ELEVATION: Green <25%, Yellow 25-50%, Red >50%
  - TRUNK BENDING 20-45°: Green <12.5%, Yellow 12.5-25%, Red >25%
  - TRUNK BENDING >45°: Green <6%, Yellow 6-12.5%, Red >12.5%
- Request: Actionable recommendations

Languages: Fully localized prompts in EN, SV, DE, ES, NL

Response: Stored in `session.ai_report` JSONField

Tracking: Each request creates an `AIRequestLog` record

Session Merge

`MergeSessions` view:

- Accepts list of session IDs
- Combines: calculations, tno_calculations, heart_rate_calculations, temperature_calculations, tasks data
- Merges events and devices_json
- Merges measurement files from Firebase Storage
- Creates single combined session

> `flag: Does not use transaction.atomic() — partial merge possible on failure.`

2.5 Authentication & Authorization

Firebase Authentication Backend

`FirebaseAuthentication` (DRF auth class):

```
def authenticate(request):
    1. Extract token from Authorization: Bearer <token>
    2. firebase_admin.auth.verify_id_token(token)
    3. Extract Firebase UID from decoded token
    4. Get or create User where user_name=uid
    5. Update user.last_seen
    6. Return (user, token)
```

> `flag: Line 49 has bare except: clause — catches all exceptions silently.`

Custom Firebase Claims

Set on User `pre_save` signal:

```
firebase_admin.auth.set_custom_user_claims(uid, {
    'role': user.role,
    'organization_id': user.organization_id
})
```

Permission Classes

CLASS	ACCESS
`LevelZeroPermission`	WERGONIC_ADMIN only, or allowlisted users
`LevelOnePermission`	ORG_ADMIN or ERGONOMIST in active organization
`SessionCrudPermission`	Session owner (worker) OR ORG_ADMIN/ERGONOMIST in same org OR superuser
`IsWergonicAdminOrReadOnly`	WERGONIC_ADMIN = full, others = read-only

Magic Link Implementation

1. POST email to `/users/auth/passwordless-signin/` (or mobile variant)
2. Backend calls `firebase_admin.auth.generate_sign_in_with_email_link(email, action_code_settings)`
3. ActionCodeSettings points to frontend callback URL
4. Link URL-encoded and appended to callback: `{FRONTEND_URL}/auth/process-login?link={encoded_link}`
5. Email sent with link
6. Frontend receives link, calls `firebase.auth.signInWithEmailLink(email, link)`
7. Firebase returns ID token

Throttling

- `custom_post`: 10 requests/minute (only on specific endpoints)
- No global throttle on auth endpoints

2.6 Firebase Integration

Firestore Document Structure

```
organizations/
  {org_id}/
    sessions/
      {session_id}/           # Active session status doc
        status, worker_id, started_at...
        (DELETED when session finishes — pre_save signal)
    users/
      {user_id}/
        notifications/
          {notification_id}/  # Push notification docs
            title, message, type, created_at
```

Firebase Storage Paths

CONTENT	PATH PATTERN
Session CSV ZIP	`organizations/{orgId}/sessions/{sessionId}/zip/session.zip`
Session logs media	`Logs/{sessionId}/{filename}`
Measurements	`Measurments/{username}/{filename}`
Org logos	`/logos/{orgName}/{filename}`
Firmware files	`/firmware/{version}/{filename}`

Signed URLs: Generated with 2-hour expiration (7200s) for media access in LogSerializer.

Firebase Admin SDK Usage

- `firebase_admin.auth` — Token verification, user management, custom claims, sign-in links
- `firebase_admin.firestore` — Document CRUD for sessions and notifications
- `firebase_admin.storage` — File upload/download, signed URL generation

2.7 Background Tasks & Scheduling

Thread-Based Processing

Session calculations run in a Python `Thread` spawned from `post_save` signal:

```
def Session_post_save(sender, instance, **kwargs):
    if instance.status == 'FINISHED' and not instance.calculations_completed:
        thread = Thread(target=process_session, args=(instance.id,))
        thread.start()
```

> `flag: No task queue (Celery not configured despite being a dependency). No retry logic, no monitoring, no dead-letter queue. Thread silently dies on exception.`

Cron Jobs (django-crontab)

```
CRONJOBS = [
    ("* * * * *", "python manage.py session_check >> /tmp/session_check.log 2>&1"),
    ("* * * * *", "python manage.py session_status_check >> /tmp/session_status_check.log 2>&1"),
]
```

- `session_check` : Runs every minute — monitors session states
- `session_status_check` : Runs every minute — checks session processing status

> `flag: Cron commands use hardcoded /usr/local/bin/python3 path.`

Frontend / Web Applications

3.1 Project Structure

Monorepo: Turborepo + Yarn workspaces

```
wergonic-web-apps/
├── apps/
│   ├── client-panel/          # Main client-facing app
│   │   ├── src/
│   │   │   ├── api/           # 10 files — API endpoint definitions
│   │   │   ├── atoms/         # 6 files — Jotai state atoms
│   │   │   ├── components/    # Shared components
│   │   │   ├── features/      # 13 feature modules
│   │   │   │   ├── auth/
│   │   │   │   ├── RAMP/
│   │   │   │   ├── RULA/
│   │   │   │   ├── REBA/
│   │   │   │   ├── MEC/
│   │   │   │   ├── sessions/
│   │   │   │   ├── statistics/
│   │   │   │   ├── dashboard/
│   │   │   │   ├── workcycles/
│   │   │   │   ├── users/
│   │   │   │   ├── workProfiles/
│   │   │   │   └── CategoriesAndLabels/
│   │   │   ├── hooks/         # 7 custom hooks
│   │   │   ├── pages/         # 31 page components
│   │   │   ├── providers/     # Global providers
│   │   │   ├── routes/        # Routing config
│   │   │   ├── services/      # Firebase, Axios
│   │   │   ├── types/         # 15 type definition files
│   │   │   └── utils/         # i18n, utilities
│   │   └── vite.config.ts
│   └── wergonic-admin/        # Admin panel
│       ├── src/
│       │   ├── api/           # 6 files
│       │   ├── atoms/         # Auth + theme
│       │   ├── features/      # 6 feature modules
│       │   │   ├── auth/
│       │   │   ├── dashboard/
│       │   │   ├── organizations/
│       │   │   ├── users/
│       │   │   ├── devices/
│       │   │   └── firmware/
│       │   ├── hooks/
│       │   ├── pages/         # 4 pages
│       │   ├── providers/
│       │   ├── routes/
│       │   ├── services/
│       │   └── types/
│       └── vite.config.ts
├── packages/
│   ├── ui/                    # Shared UI component library
│   ├── eslint-config-custom/  # Shared ESLint config
│   └── tsconfig/              # Shared TS config
├── turbo.json
└── package.json
```

Total: ~1,599 TypeScript files in client-panel alone

3.2 State Management

Jotai (Atomic State)

Key atoms:

```
// Auth
authAtom            // Current Firebase User + WergonicUser
pendingAuthStateAtom // Auth initialization flag

// Feature flags
featureFlagsAtom    // { enable_TestingNewComponentsPage: DEV, enable_IconsPage: DEV }

// UI
themeAtom           // Dark/light theme toggle
isDrawerOpenAtom    // Mobile drawer state
settingsModalOpenAtom // Modal visibility

// Data
selectedAssessmentRowsAtom // Multi-row selection in assessment tables
```

TanStack React Query (Server State)

Configuration:

```
new QueryClient({
  defaultOptions: {
    queries: {
      keepPreviousData: true,
      retry: false,
      refetchOnWindowFocus: false,
      staleTime: 1000 * 60 * 15,  // 15 minutes
    },
  },
})
```

Key Query Hooks:

- `useRetrieveActiveOrganizationStatsQuery()` — Dashboard statistics
- `useRetrieveSessionQuery()` — Single session data
- `useListOrganizationSessionsQuery()` — Session list with filters
- `useListAssessmentsQuery()` — Assessment list
- `useSingleAssessmentQueryCache()` — Cached assessment
- `useGetAssessmentInfoFormatedData()` — Formatted assessment display
- `useListMultiAssessmentsQuery()` — Multiple assessment comparison

3.3 Component Architecture

Shared UI Library (`packages/ui/`)

- `GlobalLoader` — Loading skeleton
- `PaginatedTableV2` — Reusable paginated table with sorting/filtering

- `Tabs/TabPanel` — Tab navigation
- Chart wrappers (Nivo): ResponsiveLine, ResponsiveBar, ResponsivePie
- Form components: Input, Select, Checkbox
- Icon library: MUI icons + custom
- Theme definitions: `lightTheme`, `darktheme`

Styling Approach

- **Primary:** Material-UI 5 with Emotion styled-components
- **Pattern:** `styled()` from `@mui/material/styles` + `sx` prop
- **Theme:** Custom color system with `color_system.text.primary`, `divider`, `status.infos`, etc.
- **Responsive:** MUI Box with responsive sx values + Flexbox layouts
- **Global:** CssBaseline from MUI for CSS reset

3.4 API Integration

HTTP Client: Axios

Setup:

```
// Request interceptor
axios.interceptors.request.use(async (config) => {
    const user = firebase.auth().currentUser;
    if (user) {
        const token = await user.getIdToken();
        config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
});
```

Base URL: `${VITE_API_BASE_URl}${VITE_API_VERSION}` (e.g., `https://api.wergonic.com/v1/`)

Error Handling: Minimal — `Promise.reject` returned, component-level handling via React Query error states.

Standard Paginated Response:

```
{
  results: T[],
  count: string,
  next: string,
  previous: string,
  num_pages: number
}
```

3.5 Charts & Visualization

Library: Nivo 0.83.0

CHART TYPE	COMPONENT	DATA SOURCE	FEATURES
Session time-series	PaginatedLineChart (ResponsiveLine)	`/sessions/{id}/stats/`	Zoom, pagination, downsampling, log event markers, custom tooltip
Distribution donut	DonutChartCard (ResponsiveDonut)	Session stats	Custom center content, tooltip
Temperature trend	ResponsiveLine	`/sessions/{id}/stats/temperature/`	Per-limb time series
Heart rate trend	ResponsiveLine	`/sessions/{id}/stats/hr/`	BPM + HRV
Percentile distribution	ResponsiveLine	`/sessions/{id}/stats/percentiles/`	Statistical distribution
Multi-assessment compare	Bar charts + tables	`/assesments/multi/?ids`	Side-by-side

Data Transformation:

- Large datasets downsampled via `downsample()` utility
- Color coding from theme system
- Custom axis label formatting via date-fns

Mobile App (Flutter)

4.1 Project Structure

Package: `ergotech` v4.0.2+273 (build 273) **Architecture:** Clean Architecture with feature-based modules

```
lib/
├── features/
│   ├── auth/
│   │   ├── domain/     (entities, repos, usecases)
│   │   ├── data/       (models, datasources, repo impls)
│   │   └── presentation/ (pages, cubits, widgets)
│   ├── bluetooth/
│   ├── measurements/
│   ├── sessions/
│   ├── settings/
│   ├── work_cycle/
│   ├── work_labels/
│   ├── tickets/
│   └── internet_connectivity/
├── core/
│   ├── database/       (Drift DB, DAOs, migrations)
│   ├── constants/      (app constants, BLE paths)
│   ├── enums/          (limbs, roles, sync status)
│   ├── errors/         (exception types)
│   ├── functions/      (utilities)
│   ├── network/        (Dio client, interceptors)
│   ├── services/       (Firebase, analytics)
│   └── utils/          (formatting, conversions)
└── main.dart
```

Dependency Injection: GetIt + injectable (annotation-based registration)

4.2 State Management

Pattern: BLoC/Cubit (flutter_bloc v8.1.2)

All state managed via Cubit (Bloc without events):

- State classes generated with `freezed`
- Transitions via `.emit(newState)`
- Stream-based for real-time UI updates

Key Cubits

CUBIT	PURPOSE	KEY STATES
`LoginCubit`	Auth flow	initial, loading, success, error
`UserCubit`	Current user	initial, loaded(user), error
`UserProfileCubit`	Profile CRUD	initial, loaded, creating, error
`OrganizationCubit`	Org list/switch	initial, loaded(orgs), switching
`SensorConnectionCubit`	BLE device management	disconnected, scanning, connecting, connected(Map<Limbs, Device>), reconnecting
`SensorScanCubit`	BLE discovery	initial, scanning(devices), stopped
`SensorBatteryCubit`	Per-device battery	tracking(Map<Device, int>)
`BluetoothAdapterStatusCubit`	BLE on/off	on, off, unavailable
`WorkSessionCubit`	Active session	initial, created, recording, paused, ended, syncing, synced, error
`SessionTimerCubit`	Elapsed time	ticking(seconds)
`SessionEventCubit`	Task/break events	initial, events(list)
`SessionHistoryCubit`	Past sessions	initial, loaded(sessions)
`CalibrationCubit`	Calibration progress	initial, ongoing(step), complete(data), error
`InternalMeasurementsCubit`	Phone sensor data	initial, measuring(data), error
`MeasurementsManagerCubit`	Coordinates all measurements	initial, streaming, stopped
`ReportCubit`	Session analysis	initial, computing, loaded(report), error
`WorkSessionListingCubit`	Session CRUD	initial, loaded(sessions), error
`SelectableLimbsCubit`	Limb selection	loaded(Map<Limb, bool>)
`SettingsCubit`	All settings	loaded(settings)
`DevModeCubit`	Developer features	enabled/disabled
`WorkCategoriesCubit`	Task categories	loaded(categories)
`WorkCycleCubit`	Work cycle models	loaded(hierarchy)
`InternetCubit`	Connectivity	online, offline
`TicketsCubit`	Issue reporting	initial, submitting, submitted
`LocationMeasurementsCubit`	GPS tracking	initial, tracking(locations)

4.3 Local Database (Drift/SQLite)

Engine

- **Library:** Drift v2.28.2 (code-generated SQLite ORM)
- **File:** `ergonomy_db.db` in app documents directory
- **Generated code** via `build_runner`

Complete Schema

Auth & User Tables:

TABLE	FIELDS
`ConsultantTable`	id (PK), name, email, phone
`ProfileTable`	id (PK), userId (FK), name, role, organization, createdAt
`UserTable`	id (PK), uid (unique), email, firstName, lastName, role (enum), organization (FK), active, createdAt, updatedAt

Bluetooth Devices:

TABLE	FIELDS
`BleDevicesTable`	id (PK), localId (unique), serial, name, mac, limb (enum), firmware (FK), status (enum: connected/disconnected), pairedAt, lastConnectedAt, battery (%), metadata (JSON)
`FirmwareTable`	id (PK), deviceId (FK), version, releaseDate, filename, fileSize, checksum, downloadUrl, installed (bool)

Sessions & Recording:

TABLE	FIELDS
`WorkSessionsTable`	id (PK), userId (FK), workerProfileId (FK), remoteSessionId (FK: unique), firstName, lastName, gender (enum), dominantArm (enum), age, weight, workplace, organizationId (FK), comment, syncStatus (enum), startedAt, createdAt, endedAt
`SessionEventsTable`	id (PK), sessionId (FK), eventType (enum: taskEvent/breakEvent), eventName, phase (enum: start/finish), timestamp
`SessionDevicesTable`	id (PK), sessionId (FK), deviceId (FK), limb (enum), connectedAt, disconnectedAt, samplingRate
`TimeIntervalsTable`	id (PK), sessionId (FK), startTime, endTime, riskLevel (enum)
`TimePercentageTable`	id (PK), sessionId (FK), riskLevel (enum), percentage, minutes

Risk Data (computed):

TABLE	FIELDS
`ArmAngularVelocityRiskDataTable`	id (PK), reportId (FK), limb, avgVelocity, maxVelocity, occurrences, percentage, riskLevel
`ArmElevationAngleRiskDataTable`	id (PK), reportId (FK), limb, time0_30, time30_60, time60_90, percentage, riskLevel
`TrunkBendingRiskDataTable`	id (PK), reportId (FK), bendingType (forward/side), angle20_45, angle45_plus, percentage, riskLevel
`HeadBendingRiskDataTable`	id (PK), reportId (FK), pitchAngle, rollAngle, percentage, riskLevel
`HrRiskDataTable`	id (PK), reportId (FK), avgHr, maxHr, restingHr, percentage, riskLevel

Reports:

TABLE	FIELDS
`SessionReportTable`	id (PK), sessionId (FK), remoteReportId, duration, totalTime, activeTime, riskScore, postureScore, movementScore, reportData (JSON)
`ReportDataTable`	id (PK), reportId (FK), metric, value, threshold, status
`TNOReportTable`	id (PK), sessionId (FK), remoteReportId, nmsScore, riskLevel, feedback (JSON), timestamp

Work Cycles:

TABLE	FIELDS
`WorkCycleFactoryTable`	id, factoryId, name, location, remoteId
`WorkCycleLineTable`	id, lineId, factoryId (FK), name, remoteId
`WorkCycleModelTable`	id, modelId, lineId (FK), name, active, remoteId
`WorkCycleStationTable`	id, stationId, modelId (FK), stationName, remoteId
`WorkCycleTasksTable`	id, taskId, stationId (FK), taskName, cycleTime, remoteId

Configuration:

TABLE	FIELDS
`WorkCategoryTable`	id, categoryId, organizationId (FK), categoryName, categoryType (enum), remoteId
`WorkLabelTable`	id, labelId, categoryId (FK), labelName, color, remoteId
`SettingsTable`	id, userId (FK), liteMode, isHrActive, feedbackType, voiceFeedbackIntervals, feedbackRefreshRate, gyroRange, accRange, integralGainValue, proportionalGainValue, standardImuRate, refImuRate, headSensorPosition, workSessionType, timeBasedWorkCycles, newWorkCycleFrequency, sessionDurationLimit, tnoArchiveGeneration, appLanguage, offlineMode, limbsFeedbackOrder, isTwoPointCalibrationEnabled, isVibrationDetectionEnabled, batteryCheckIntervalSeconds, autoStartSession, enableLocation, isPerformanceOverlayEnabled, useNewCsvSystem, useImprovedVibrationDetection, confidenceThresholdHigh/Low, accThresholdHigh, gyroThresholdHigh, primaryWeight, ratioWeight, showExtraTiles
`UserSelectableLimbConfigurationTable`	id, userId (FK), limb (enum), enabled (bool), calibrationDate, calibrationData (JSON)
`OrganizationTable`	id, orgId, name, country, industry, remoteId, active, createdAt

DAO Layer

Each table has a corresponding DAO handling:

- CRUD operations
- Complex queries with joins and filtering
- Transaction management
- Relationship loading

4.4 BLE / Sensor Layer

Library Stack

LAYER	LIBRARY	VERSION	PURPOSE
System BLE	flutter_blue_plus	1.35.3	Android BLE integration, DFU mode scanning
Movesense SDK	mdsflutter (MDS)	2.3.1	Sensor communication protocol
Firmware update	nordic_dfu	6.1.2	Over-the-air firmware updates

Connection Management

Scan:

```
// Normal mode: MDS-based scan
MdsAsync.startScan((device) => emit(device));

// DFU mode: flutter_blue_plus scan
FlutterBluePlus.startScan(withNames: ["dfu"]);
```

Connect:

```
Mds.connect(
  deviceId,
  onConnect: (serial) => { /* store device */ },
  onDisconnect: () => { /* handle reconnection */ },
  onError: (error) => { /* log + retry */ },
  onData: (data) => { /* unused for connection */ }
);
```

Post-Connection BLE Parameters:

```
min_conn_interval: 39    // 48.75ms
max_conn_interval: 80    // 100ms
slave_latency: 0
supervision_timeout: 1000ms
```

Data Streaming

IMU6 Subscription:

```
MdsAsync.subscribe(
  serial,
  '/Meas/IMU6/{rate}',  // rate: 52, 104, or 200 Hz
  (data) => processImuData(data)
);
```

Raw Data Format (from Movesense):

```
{
  "Body": {
    "Acc": [x, y, z],      // Accelerometer -8 to +8G
    "Gyro": [x, y, z],     // Gyroscope -2000 to +2000 °/s
    "Ts": 1234567           // Timestamp (ms)
  }
}
```

Processing Pipeline:

1. Raw BLE JSON → `ReadingEntity` (6D: 3 accel + 3 gyro)
2. Butterworth IIR low-pass filter (cutoff ~1.5x feedback rate)
3. Mahony complementary filter (proportionalGain ~2.0, integralGain ~0.0) → quaternion (q0, q1, q2, q3)
4. Risk metrics extraction:
   - Elevation Angle: `arccos(q3) × 2` (degrees, 0-90)
   - Angular Velocity: `magnitude(gyro)` (°/s, 0-400)
   - Forward/Side Bending: roll/pitch from quaternion
   - Vibration Detection: high-frequency gyro variance + confidence scoring

MDS SDK Paths

PATH	METHOD	PURPOSE
`/System/Mode`	PUT (int: 1)	Enter DFU mode
`/Info`	GET	Device metadata (HW/FW version)
`/Info/App`	GET	App-level info
`/Comm/Ble/Peers`	GET	Connected BLE peers
`/Comm/Ble/Peers/{handle}/Params`	GET/PUT	BLE connection parameters
`/Meas/IMU6/{rate}`	SUBSCRIBE	6D IMU at rate Hz
`/Meas/HR`	SUBSCRIBE	Heart rate
`/Meas/Temp`	SUBSCRIBE	Temperature
`/Time`	GET	Device time
`/Time/Detailed`	GET	Detailed time info

Sensor Types

LIMB	CATEGORY	MEASURES	CALIBRATION
rightArm	Main	Elevation angle, angular velocity, bending	Natural + Bent
leftArm	Main	Elevation angle, angular velocity, bending	Natural + Bent
trunk	Main	Pitch, roll bending	Natural + Bent (2 points required)
head	Main	Pitch, roll bending	Natural + Bent
hand	Reference	Quaternion only	Natural only
forearm	Reference	Quaternion only	Natural only
chestStrap	HR	Heart rate BPM	None

4.5 Session Recording Engine

CSV File Generation

Per Limb CSV:

```
Filename: measurements_{limb}_{workerName}_{datetime}.csv
Columns: timestamp, elevationAngle, angularVelocity, gyroBasedAngularVelocity,
         pitch, roll, yaw, accX, accY, accZ, gyroX, gyroY, gyroZ,
         quaternion_w, quaternion_x, quaternion_y, quaternion_z,
         vibrationResult, averageHr
```

HR File:

```
Filename: measurements_HR_{workerName}_{datetime}.csv
Columns: timestamp, heartRate, confidence
```

Location File:

```
Filename: location_{sessionId}_{datetime}.csv
Columns: timestamp, latitude, longitude, accuracy, altitude, speed
```

CSV Writer Architecture

CsvWriterManager:

- One `LimbCsvWriter` per active limb
- In-memory write queue per limb
- Periodic flush: every 500ms
- Batch writes to disk for performance
- Error stream for monitoring write failures

Legacy System (fallback):

- `WriteDataToCsvUseCase.writeBuffer`: `List<ProcessedDataEntity>`
- Max buffer size: `Constants.bleDataBufferSize`
- Flushes when buffer full or session ends
- Toggle via `enableNewCsvSystem` setting

Session ZIP & Upload

```
session_measurements_{sessionId}_{date}.zip
├── measurements_rightArm_{worker}_{datetime}.csv
├── measurements_leftArm_{worker}_{datetime}.csv
├── measurements_trunk_{worker}_{datetime}.csv
├── measurements_HR_{worker}_{datetime}.csv
└── location_{sessionId}_{datetime}.csv (optional)
```

4.6 Sync Service

Sync Flow (`syncWorkSession`)

```
Step 1: Create remote session
  POST /sessions/organizations/{orgId}/
  Body: WorkSessionModel.toJson()
  → Receives remoteSessionId
  → Update DB: syncStatus → createdNotSynced

Step 2: Create ZIP
  Read all CSV files from session.path
  Compress into single ZIP

Step 3: Sync work tasks/events
  POST /categories/labels/events/sessions/{sessionId}/bulk_create/
  Body: list of SessionEventEntity

Step 4: Upload to Firebase Storage
  FirebaseStorage.ref("organizations/{orgId}/sessions/{sessionId}/zip/session.zip")
    .putFile(zipFile)
  → Real-time progress: bytesTransferred / totalBytes

Step 5: Finalize session
  PATCH /sessions/{sessionId}/
  Body: { tasks: session.eventsToList, ended_at: ISO8601 }
  → Update DB: syncStatus → fullySynced
```

Sync Status Progression

```
notSynced → createdNotSynced → fullySynced
                                    ↑
historyNotSynced ———————————————————┘
```

No Queue Persistence

- Sessions marked `createdNotSynced` are implicitly "in queue"
- No persistent job queue
- Retry: manual user action or background check on reconnect
- Sequential sync (one session at a time)

4.7 Offline Architecture

Caching Strategy

DATA	CACHED	TTL	REFRESH TRIGGER
Work Cycles	Yes (DB)	None	Online reconnect
Work Categories	Yes (DB)	None	`CheckWorkCategoriesExistanceUsecase`
Sensor Metadata	Yes (DB)	Permanent	After pairing
Calibration Data	Yes (DB)	Permanent	User recalibrates
Sessions	Yes (DB)	Permanent	User deletes
Settings	Yes (DB)	Permanent	User changes

Data Priority

1. During session: Always local sensors (no remote fallback)
2. Post-session: Local data uploaded, remote DB becomes source of truth
3. Work cycles: Local cache if offline, remote if online
4. User data: Local cache until token expires

4.8 Platform Channels & Native Code

Android (Kotlin)

MainActivity.kt (`com.wergonic.wergo.MainActivity`):

CHANNEL	METHOD	PURPOSE
`sensor_availability`	`isGyroAvailable`	Check hardware gyroscope exists
`sensor_availability`	`isAccelerometerAvailable`	Check hardware accelerometer exists
`performance_monitor`	`getMetrics`	FPS, memory, render time
`performance_monitor`	`resetMetrics`	Reset performance counters

Background Service:

- Foreground notification: "Recording in progress"
- Wakelock + WorkManager for sustained recording
- Battery optimization exemption

iOS

- Minimal native code — Flutter defaults used
- No custom platform channels observed

Data Pipeline (End to End)

Complete Flow: Sensor → Screen

```
Step 1: SENSOR READING
  Movesense IMU6 sensor → raw accelerometer + gyroscope data
  Format: {Acc: [x,y,z], Gyro: [x,y,z], Ts: ms}
  Rate: 104 Hz (default)

Step 2: BLE TRANSMISSION
  Movesense → BLE → MDS SDK → Flutter callback
  Protocol: Movesense proprietary over BLE GATT
  Latency: ~10-50ms per packet

Step 3: SIGNAL PROCESSING (on phone)
  Raw data → Butterworth IIR filter (noise removal)
  Filtered data → Mahony complementary filter → quaternion
  Quaternion → elevation angle, angular velocity, bending angles
  Output: ProcessedDataEntity per sample

Step 4: LOCAL STORAGE (on phone)
  ProcessedDataEntity → CSV buffer (in memory)
  Buffer flush → CSV file on device storage (every 500ms)
  Session end → ZIP all CSVs

Step 5: UPLOAD TO FIREBASE STORAGE
  ZIP file → FirebaseStorage.ref("organizations/{orgId}/sessions/{sessionId}/zip/")
  Real-time upload progress tracking

Step 6: BACKEND PROCESSING (Django)
  Session post_save signal → Thread spawned
  Download ZIP from Firebase → extract CSVs
  Parse CSV → create MeasurementData records (TimescaleDB)
  Rate: bulk insert, interval="8 hours" hypertable chunks

Step 7: STATISTICAL CALCULATIONS
  MeasurementData → posture percentages, movement distributions
  MeasurementData → temperature aggregations (mean/max/min/median)
  HRData → HRV analysis (SDNN, RMSSD, LF/HF, CSI/CVI)
  MeasurementData → chart data (line, pie, bar)
  MeasurementData → TNO analysis (static/dynamic, angle ranges)

Step 8: RESULTS STORAGE
  All calculations → Session JSONFields
  Chart data → SessionStats model

Step 9: AI ANALYSIS (optional)
  Session data → Gemini prompt (localized)
  Gemini response → Session.ai_report JSONField

Step 10: FRONTEND DISPLAY
  React Query fetches → /sessions/{id}/stats/
  Stats JSON → Nivo charts (line, pie, donut)
  Assessment data → form views, results pages
  AI report → formatted text display
```

Data Formats at Each Step

STEP	FORMAT	SIZE (TYPICAL)
Raw IMU	JSON per packet	~200 bytes
Processed	ProcessedDataEntity	~150 bytes
CSV per limb	CSV file	5-50 MB (8-hour session)
ZIP per session	Compressed ZIP	10-100 MB
MeasurementData rows	TimescaleDB	~3M rows (8h, 104Hz)
Stats JSON	JSONField	10-100 KB
AI Report	JSONField	2-10 KB

Where Data Can Be Lost

POINT	RISK	MITIGATION
BLE disconnect during recording	Data gap	Reconnection (up to 3 retries)
CSV buffer crash	Unbuffered data lost	Periodic flush (500ms)
Upload failure	Session stays local	Manual retry
Backend processing failure	Session unprocessed	**None — no retry mechanism**
Firestore doc deletion	Active session status lost	Pre-save signal deletes on FINISH only

Third-Party Integrations

SERVICE	USAGE	CONFIGURATION
Firebase Auth	User authentication, token management, magic links, custom claims	Per-environment credentials
Cloud Firestore	Real-time session status, push notifications	Per-environment project
Firebase Storage	CSV/ZIP files, org logos, firmware files, session media	Per-environment bucket
Firebase Analytics	User events, session tracking	Auto-configured
Firebase Crashlytics	Mobile crash reporting	Auto-configured
Google Gemini 2.0	AI-powered ergonomic report generation	`GEMINI_API_KEY`
Movesense MDS SDK	BLE sensor communication protocol	Bundled in app
Nordic DFU	Sensor firmware updates	Bundled in app
Sentry	Error tracking (web + mobile)	Separate DSNs per platform
Slack	Backend notifications	`slack_token`
SMTP Email	Transactional emails (invitations, magic links, password reset)	`EMAIL_HOST/USER/PASSWORD`, port 465 SSL
reCAPTCHA v3	Firebase AppCheck (web)	`VITE_RECAPTCHA_SITE_KEY`

Deployment & Infrastructure

Backend

- **Server:** Gunicorn 22.0.0 (WSGI)
- **Database:** PostgreSQL + TimescaleDB extension
- **Environment config:** python-decouple with `.env` file
- **Django Channels:** 4.1.0 installed (ASGI capability) but no WebSocket consumers implemented

Web Apps

- **Build:** Vite 4.1.1 + Turborepo
- **Docker:** Multi-stage builds, Docker Compose
- **Ports:** 443 (HTTPS), 8443 (secondary)
- **Max build memory:** 8192 MB

Mobile App

- **Build:** Flutter SDK ≥3.3.1
- **CI/CD:** Codemagic (inferred from mobile build pipeline)
- **Platforms:** Android (primary, native code present), iOS (Flutter defaults)
- **Version:** 4.0.2+273

Monitoring

- **Sentry** — Error tracking for web (DSN: `...ingest.sentry.io/4504765616750592`) and mobile (separate DSN)
- **Firebase Crashlytics** — Mobile crash reports
- **Firebase Analytics** — Usage analytics
- **Slack** — Backend event notifications

Technical Debt Register

Critical Issues

1. Background Processing Without Task Queue

- **Location:** `user_sessions/` signals
- **Problem:** Session calculations run in Python `Thread` from Django signal — no Celery/task queue
- **Impact:** No retry on failure, no monitoring, no dead-letter queue. Processing thread silently dies on exception. Session stays unprocessed permanently.
- **Fix:** Migrate to Celery with Redis/RabbitMQ broker

2. Security: ALLOWED_HOSTS = ["*"]

- **Location:** `core/settings.py:24`
- **Problem:** Accepts requests from any host in all environments
- **Impact:** Host header injection vulnerability
- **Fix:** Set per-environment allowed hosts

3. Password Emailed in Plaintext

- **Location:** `organizations/views/organizationView.py:347`
- **Problem:** Password sent via email during user creation
- **Impact:** Credential exposure
- **Fix:** Use password reset flow instead

4. Session Status Transitions Not Enforced

- **Problem:** Any status can be set to any other via API (e.g., FINISHED → ONGOING)
- **Impact:** Invalid state, potential data corruption
- **Fix:** Implement state machine validation

5. No Transaction Atomicity on Merge

- **Location:** `MergeSessions` view
- **Problem:** Session merge doesn't use `transaction.atomic()`
- **Impact:** Partial merge on failure — corrupted data
- **Fix:** Wrap in atomic transaction

High Priority

6. Bare Exception Handling

- **Location:** `firebase_auth/authentication.py:49`
- **Problem:** `except:` catches all exceptions silently
- **Fix:** Catch specific exceptions, log others

7. No Sync Retry Mechanism (Mobile)

- **Problem:** Failed uploads require manual user intervention

- **Impact:** Data loss if user doesn't notice sync failure
- **Fix:** Implement exponential backoff retry with persistent job queue

8. Organization Stats Computed on Every Call

- **Location:** `Organization` model properties
- **Problem:** `number_of_sessions_per_month` etc. run DB queries per request
- **Impact:** Slow dashboard for orgs with many sessions
- **Fix:** Cache stats, invalidate on change

9. MeasurementData Volume

- **Problem:** TimescaleDB table grows massively (3M+ rows per 8h session per limb)
- **Impact:** Query performance degradation over time
- **Fix:** Implement data retention policies, materialized views for aggregates

10. No API Rate Limiting on Auth Endpoints

- **Problem:** Only `custom_post: 10/min` exists, not on login/reset
- **Impact:** Brute-force vulnerability
- **Fix:** Add rate limiting to all auth endpoints

Medium Priority

11. Naming Inconsistencies

- `assesment_category` (typo throughout backend — "assessment" misspelled)
- `switchOranization` route (typo in mobile — "Organization" misspelled)
- `VITE_API_BASE_URl` (typo — "URI")
- `defaulPropotionalGain` (typo in mobile constants)
- Mixed snake_case/camelCase in JSON between backend and frontend

12. Commented-Out Code

- Backend: Proxy models (`workerModel.py` , `ergonomistModel.py` , `orgAdminModel.py`)
- Backend: Multiprocessing TNO approach (`user_sessions/models.py:262-275`)
- Backend: Test runner (`test_runner.py`)
- Web: Hundreds of "done" comments in `apiRoutes.ts`

13. Dead/Unused Code

- Web: `legacyApiRoutes.ts` (17KB file of old routes)
- Mobile: `CheckResetPasswordUseCase` defined but unused
- Mobile: Commented MDS imports in pubspec.yaml

14. Missing Tests

- Backend: Minimal test setup, test runner commented out
- No integration tests visible
- No end-to-end tests

15. Hardcoded Values

- TNO thresholds: `[60, 20, 10]` degrees
- Static duration: 6 seconds
- BLE connection intervals: 39/80 (48.75ms/100ms)
- Cron paths: `/usr/local/bin/python3`
- Session max duration: 480 minutes (8 hours)
- Signed URL expiry: 7200 seconds (2 hours)

16. Frontend-Backend Inconsistencies

- Web calls endpoints that may not match backend exactly (e.g., `/sessions/{id}/stats/percentiles/` — not clearly visible in backend URLs)
- Additional web endpoints (archiving settings, report templates, word export) not all visible in backend URL config

17. N+1 Query Issues

- Session list serializes all logs per session
- No select_related/prefetch_related optimization visible

18. Magic Link Security

- URL-encoded in query parameter — logged by proxies, browser history
- Should use short-lived tokens with server-side validation

19. Incomplete Mobile Features

- Vibration detection display (UI TODO)
- Firmware update flow (dialog incomplete)
- Sensor unpair/pair/disconnect buttons (not implemented)
- Work cycle auto-advance (timer exists, UI not wired)

20. No Caching Layer

- No Redis/memcached configured
- Django ORM hits DB on every request
- React Query client-side cache (15min) provides some relief

Low Priority

21. Missing Error Boundaries (Web)

- Global boundary exists but no granular feature-level boundaries
- API errors may crash pages

22. Console Statements in Production Code

- `TestingNewComponentsPage:12` — `console.log(allDataIsValid)`
- `auth.ts:61` (admin) — `console.error` instead of logging service

23. Feature Flags Hardcoded

- Only 2 flags, both tied to `import.meta.env.DEV`
- No remote feature flag system

24. No API Versioning Strategy

- Single `/api/v1/` prefix
- No migration path to v2

25. Null/Blank Inconsistency (Backend)

- Many CharField fields with both `null=True` and `blank=True`
- Should follow Django convention: CharField uses `blank=True` only, null for non-string fields

TODO/FIXME/HACK Items

Backend

FILE	ISSUE
`Session_post_save` signal	Thread-based processing without queue
`firebase_auth/authentication.py:49`	Bare except clause
`user_sessions/models.py:262-275`	Commented multiprocessing for TNO
`settings.py:295-296`	Hardcoded cron python path

Mobile

FILE	ISSUE
`sensor_vibration_list_item.dart:122,137,149`	Unpair/Pair/Disconnect not implemented
`firmware_update_available_dialog.dart:25`	Update flow incomplete
`work_label_model.dart:25`	Replace name with ID mapping
`switch_organization_section.dart:34`	Review code quality
`sensors_settings_section.dart:208`	Restore deferred feature
`feedback_display_panel.dart:123`	Move logic elsewhere
`vibration_detection_panel.dart:65,85`	Time display + limb toggle
`session.remote.datasource.dart:112,146`	Review logic + error handling
`work_session_listing.page.dart:87`	Change cubit initialization
`work_session_setup.page.dart:24`	Review code
`category_with_label_search_widget.dart:136`	Selection logic unclear
`firmware.remote.datasource.dart:39`	Add error handling
`sensors_scanning_dialog.dart:78`	Verify vibrator detection

Web

FILE	ISSUE
`apiRoutes.ts:1` (client-panel)	"todo delete all comments"

Dependencies Analysis

Backend (166 packages)

Core Framework:

- Django 5.0.9, DRF 3.15.2

Data Processing:

- numpy, pandas, scipy — numerical computation
- hrvanalysis — heart rate variability analysis
- astropy — scientific computing
- nolds — nonlinear dynamics analysis

Firebase:

- firebase-admin 6.1.0

AI:

- google-generativeai 0.7.2 (Gemini)

Database:

- django-timescaledb — TimescaleDB support
- psycopg2 — PostgreSQL adapter

Infrastructure:

- channels 4.1.0 — WebSocket (installed but unused)
- gunicorn 22.0.0 — WSGI server
- sentry-sdk 2.8.0

File Handling:

- openpyxl — Excel export
- django-storages 1.14.3 — cloud storage

Tree Structure:

- django-mptt — Modified Preorder Tree Traversal

Dev Tools:

- black 24.4.2, pytest 8.2.2

Web Apps

Core:

- React 18.2.0, TypeScript 4.9.3, Vite 4.1.1
- Material-UI 5.11.9, Emotion (styled)

State:

- Jotai 2.0.2, TanStack React Query 4.24.10

Charts:

- @nivo/core, @nivo/line, @nivo/bar, @nivo/pie 0.83.0

Forms:

- Formik 2.2.9, Yup 1.0.0

Data:

- Axios 1.3.4, Firebase 9.17.1
- xlsx 0.18.5, file-saver 2.0.5
- @react-pdf/renderer 3.1.12

Routing:

- react-router-dom 6.8.1

i18n:

- i18next 22.4.9, react-i18next 12.1.5

Build:

- Turborepo 1.8.1, Yarn 1.22.19

Mobile (Flutter)

Core:

- Flutter SDK ≥3.3.1, flutter_bloc 8.1.2

BLE:

- flutter_blue_plus 1.35.3, mdsflutter 2.3.1, nordic_dfu 6.1.2

Database:

- drift 2.28.2

Network:

- dio 5.3.0

Firebase:

- firebase_core, firebase_auth, cloud_firestore, firebase_storage, firebase_analytics, firebase_crashlytics

Code Generation:

- freezed 2.3.2, json_serializable 6.6.1, injectable 2.1.0

Sensors:

- sensors_plus (custom fork), flutter_activity_recognition 4.0.0

Media:

- video_player 2.9.2, audioplayers 6.1.0, image_picker 1.0.2

Export:

- pdf 3.10.8, printing 5.11.0

i18n:

- easy_localization 3.0.1

DI:

- get_it 8.0.2

Monitoring:

- sentry_flutter 8.14.1

Wergonic — internal documentation. Generated from codebase analysis of the `dev` branch (backend, web apps, mobile) on 17 March 2026.